



Data Management

Maurizio Lenzerini

***Dipartimento di Informatica e Sistemistica “Antonio Ruberti”
Università di Roma “La Sapienza”***

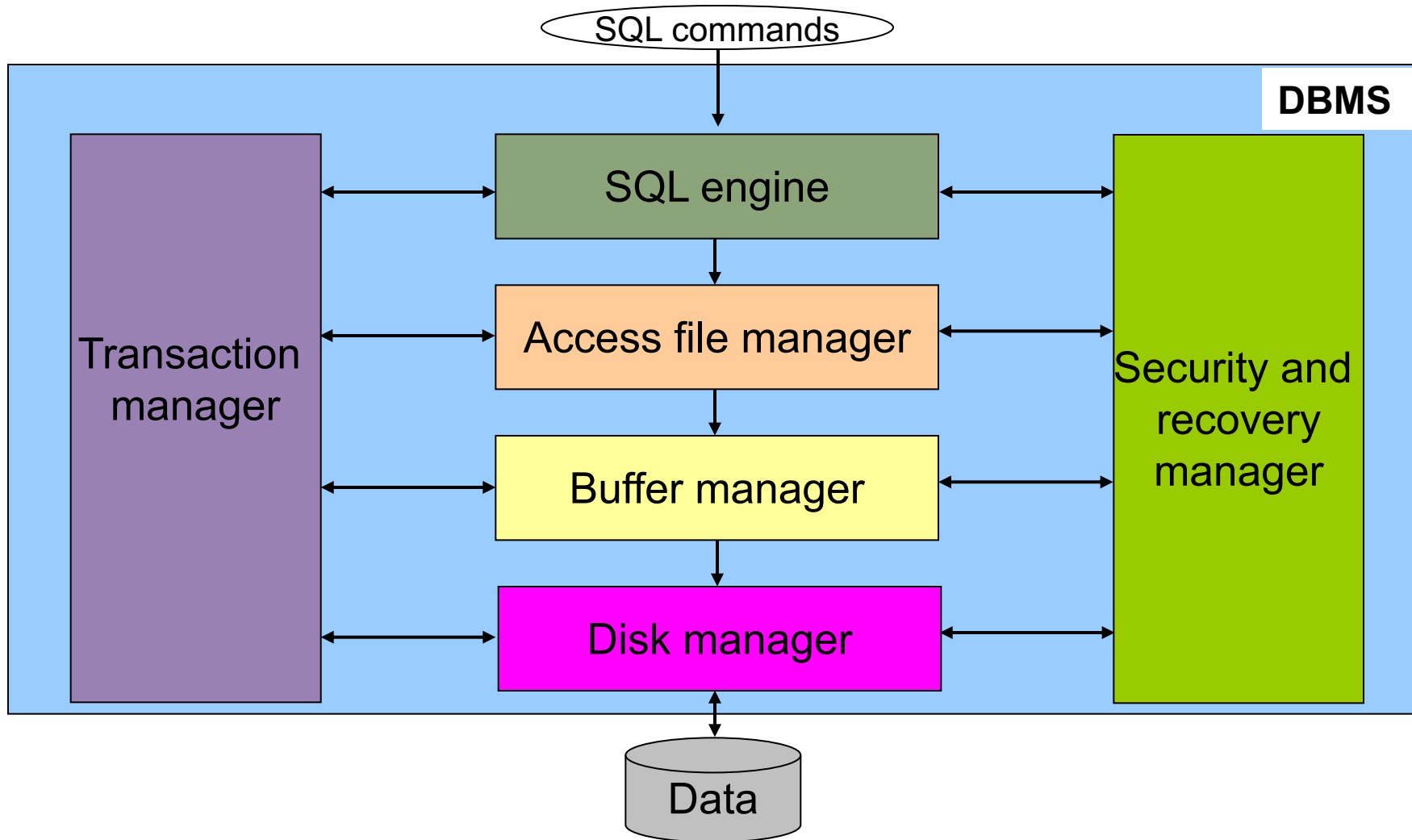
Academic Year 2023/2024

Part 6
Buffer management

<http://www.dis.uniroma1.it/~lenzerin/index.html?q=node/53>



Architecture of a DBMS





2. Buffer management



The secondary storage

At the physical level, a data base is a set of **database files**, where each file is constituted by a set of **pages**, stored in physical blocks.

Using a page requires to bring it in main memory. The size of a block (and therefore of a page) is exactly the size of the portion of storage that can be transferred from secondary storage to main memory, and back from main memory to the secondary storage.



The buffer

The **database buffer** (also called simply **buffer** or **buffer pool**) is a non-persistent main memory space used to cache database pages. Database processes request pages from the buffer manager, whose responsibility is to minimize the number of secondary memory accesses by keeping needed pages in the buffer. Because typical database workloads are I/O-bound, the effectiveness of buffer management is critical for system performance.

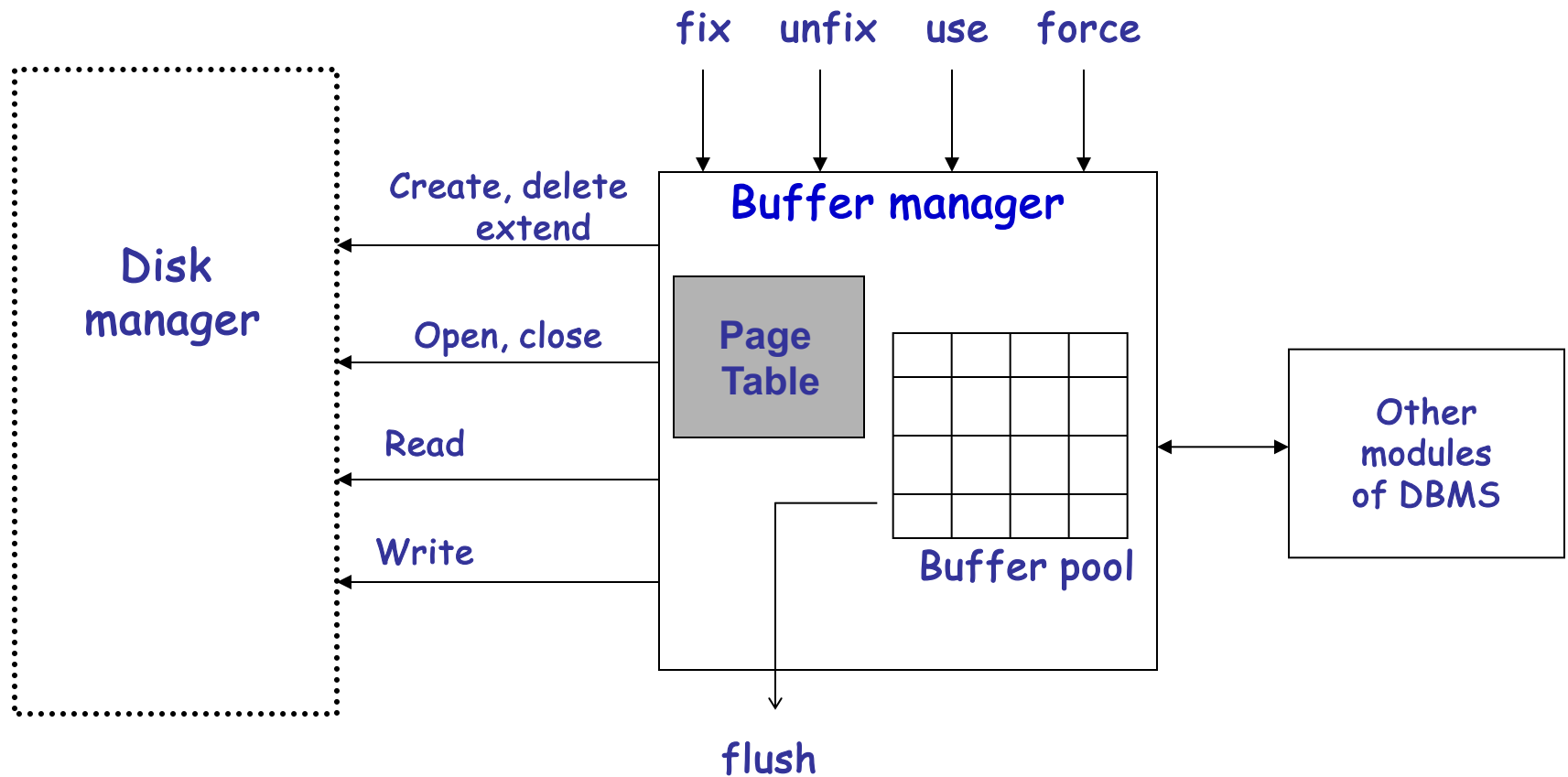
The **buffer manager** is responsible of the transfer of the pages from the secondary storage to the buffer pool, and back from the buffer pool to the secondary storage.

The buffer pool is

- Shared by all transactions (i.e., all programs using the databases managed by the DBMS)
- Used by the system (e.g., by the recovery manager)



Architecture of buffer manager





The buffer pool

- The buffer pool is organized in “frames”. Each frame has an identifier (a number), corresponds to one main memory page, whose size is that of a block, where, as we said, a block is the transfer unit from/to the secondary storage. The typical size ranges from 2Kb to 64Kb.
- Since the buffer pool is in main memory, the management of their pages is more efficient (the cost of atomic operations is of the order of billionth of seconds) with respect to the cost of the management of secondary storage pages (thousandth of seconds)
- The buffer pool is managed with similar principles as the ones used for cache memory.



The buffer manager

The buffer manager uses the “Page Table” data structure, that associates to each frame in the buffer the last secondary storage page (denoted by its Page Identifier (PID)), if any, loaded in the frame. In some sense, this frame is the “main memory twin” of such secondary storage page.

The buffer manager uses the following primitive operations:

- **Fix**: load a page
- **Unfix**: releases a page
- **Use**: uses a page in the buffer
- **Force**: synchronous transfer to secondary storage
- **Flush**: asynchronous transfer to secondary storage



The Fix operation

- An external module issues the "Fix" operation in order to ask the buffer manager to load a specific secondary storage page into the buffer
- For each frame F in the buffer, the buffer manager maintains:
 - As said, the information about which page (if any) it contains (this information is provided by the Page Table)
 - **pin-count(F)**: how many transactions are using the page contained in F ; at the beginning, it is set to 0.
 - **dirty(F)**: a bit whose value indicates whether the content of the frame F has been modified (true) or not (false) from the last load; at the beginning, it is set to false.



The Fix operation

Suppose that the DBMS module M issues a Fix operation that asks the buffer manager to load page P (denoted by its PID) in the buffer.

1. Looking at the Page Table, the buffer manager checks whether there is a frame F in the buffer pool already containing P
2. If yes, then the buffer manager increments $\text{pin-count}(F)$
3. If not, then
 1. using a certain **replacement policy**, the buffer manager chooses a frame F' (if possible) that can host page P in the buffer,
 2. If $\text{dirty}(F') = \text{true}$, then the buffer manager writes the page contained in F' back to secondary storage in order not to lose its content
 3. The buffer manager reads page P from the secondary storage and loads it in frame F'; then, it initializes $\text{pin-count}(F')$ to 0 and $\text{dirty}(F')$ to false
4. The buffer manager returns the address of the frame containing P to M



Replacement policy

The frame for the replacement (the “victim”) is chosen among those frames F with $\text{pin-count}(F) = 0$. If no frame F with $\text{pin-count}(F)=0$ exists, then the request is placed in a queue, or the transaction is aborted (and later re-executed). Otherwise, several policies are possible in order to choose the victim:

- **LRU** (least recently used): this is done through a queue containing the frames F with $\text{pin-count}(F)=0$
- **Clock replacement**:
 - We use a variable **current** pointing to frames in a circular fashion
 - Each frame F has a bit **referenced**(F) that tells us if the page contained in the frame has been used recently; more precisely, **referenced**(F) is set to true when $\text{pin-count}(F)$ is decremented to 0, to reflect the fact that, although frame F is now a candidate for replacement, it has been used recently
 - When we look for a frame for replacement, we do the following
 1. We analyze the frame F pointed by **current**
 2. If $\text{pin-count}(F)>0$, then we cannot choose the frame F pointed by **current** as a victim, and therefore we increment **current** and we repeat the procedure from step 1
 3. If $\text{pin-count}(F)=0$ and **referenced**(F)=true, we set **referenced**(F) to false, to reflect the fact that time has now passed from the last usage of the page stored in F and to let F be considered in the next round when looking for a victim, we increment **current** and we repeat the procedure from step 1
 4. If $\text{pin-count}(F)=0$ and **referenced**(F)=false, then F is chosen for replacement



Force or No-force strategy

Force/no-force:

- Force: all active pages of a transaction are written in secondary storage when the transaction commits (i.e., it ends its operations successfully)
- No-force: the active pages of a transaction that has committed are written asynchronously in secondary storage through the flush operation

Generally, the no-force is the one used, because it enables a more efficient buffer management



The other operations

- **Unfix:**
 - The transaction certifies that it does not need the content of a specific frame anymore
 - The pin-counter of that frame is decremented
- **Use:**
 - The transaction modifies the content of a frame
 - The dirty bit is set to true and, if it is the first use of the frame by the transaction, pin-count is incremented
- **Force:** Synchronous (i.e., the transactions waits for the successful completion of the operation) transfer to secondary storage of the page contained in a frame
- **Flush:** Asynchronous (i.e., executed when the buffer manager is not busy) transfer to secondary storage of the pages used by a transaction



Exercise

Suppose the buffer of our DBMS contains 10 frames, where each frame at address i contains the page whose address in secondary storage is i , all the frames with an odd address have “pin-count” >0 , all the frames with an even address have “pin-count” $=0$ and have “referenced” $=\text{true}$, and, finally, the value of “current” is 5. Assuming the clock replacement policy, describe all the actions performed by the buffer manager when the page of address 20 is asked with a fix operation, and illustrate the configuration of the buffer at the end of the execution of such operation.



Solution

When page P whose address is 20 is asked with a fix operation, since all the frames contain relevant pages, the buffer manager looks for a frame where P can be loaded. It accesses frame 5 (the one pointed by current) but all frames with odd addresses cannot be used to load P , because they have $\text{pin-count} > 0$. Therefore, the buffer manager keeps scanning the frames and since all the frames with even address have “pin-count”=0 and “referenced”=true, the buffer manager changes all such “referenced” bits to false. After frame 10, the buffer manager will go back to 1, and eventually will return to frame whose address is 6. Now, such a frame F is such that $\text{pin-count}(F)=0$ and $\text{referenced}(F)=\text{false}$, and therefore, the buffer manager will choose F as the victim, will load P in F , will modify the page table so as to register that the page in F is P , and will return 6 as the result of the fix operation. All other frames are not changed by the operation.